

Spring AI: Module 08

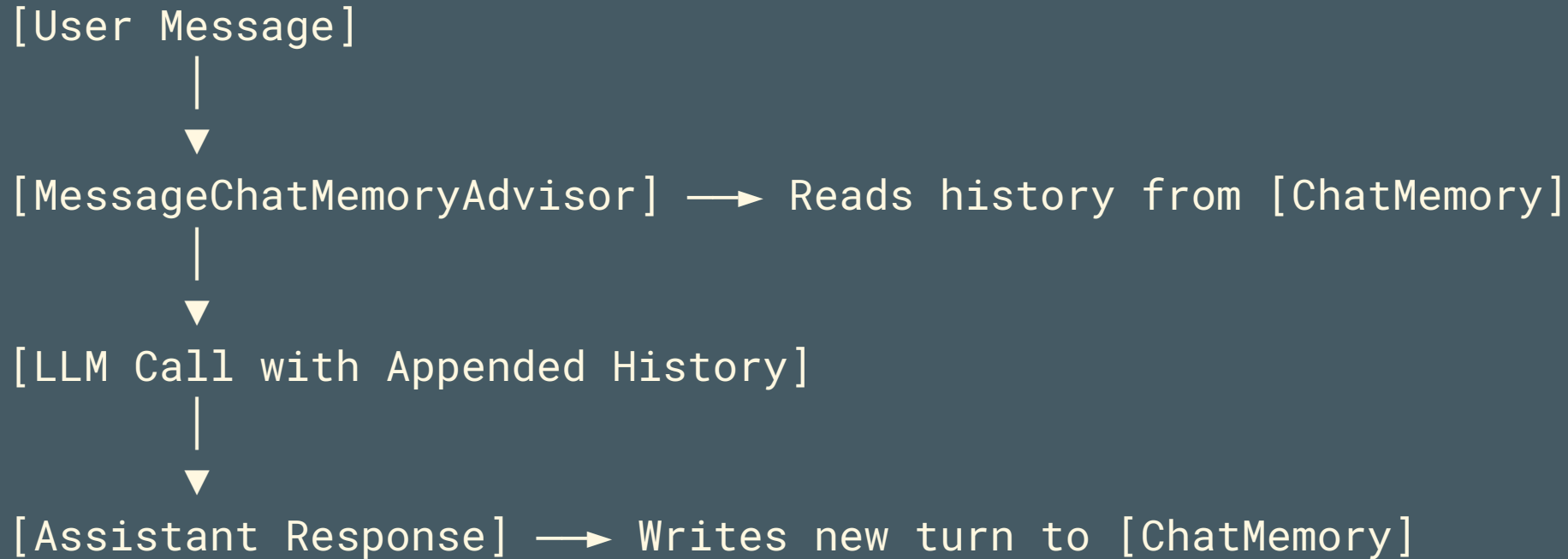
Chat Advisors, Memory Systems & Conversational State

State Management in Stateless LLM Architectures

1. The Challenge of Stateless LLMs

- HTTP and LLM provider APIs are fundamentally stateless:
 - Every API request is evaluated in total isolation.
 - The model does not remember past turns unless previous messages are sent along with each prompt.
- **Spring AI Solution:** The `ChatMemory` abstraction and the `MessageChatMemoryAdvisor`.

2. Spring AI ChatMemory Architecture



- Implementations: `InMemoryChatMemory`, Redis, JDBC, Cassandra, Neo4j.

3. Session Isolation via `conversationId`

```
String reply = chatClient.prompt()  
    .user(userMessage)  
    .advisors(a -> a.param(  
        AbstractChatMemoryAdvisor.CHAT_MEMORY_CONVERSATION_ID_KEY,  
        sessionId  
    ))  
    .call()  
    .content();
```

- Each user session stays completely isolated in memory.

4. Preventing Context Blowup: Sliding Window Token Buffers

- Without bounding, multi-turn chat history grows indefinitely:
 - Escalating token costs.
 - Exceeding LLM context window limits.
- **Custom CallAroundAdvisor:** Prunes oldest messages or summarizes conversation history when exceeding budget limits.

5. Key Architecture Takeaways

1. Never maintain conversational state in static variables; use `ChatMemory`.
2. Always isolate chat turns per unique `conversationId`.
3. Put sliding window advisors in place to cap token consumption and prevent context exhaustion.